

Міністерство освіти і науки України
Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Звіт
з лабораторної роботи No 4
з дисципліни
«Мова програмування Java»

Виконала:
студентка групи ЗПІ-зп41
Ірина Левківська

Перевірив:
Орленко С.П.

Київ 2025

Поставка завдання (Варіант 1)

Реалізувати із застосуванням узагальненого програмування ієрархію Java-класів для транспортних засобів, які можуть перевозити різні типи пасажирів.

Є наступні транспортні засоби: автобус, таксі, пожежна машина, поліцейська машина. У цих транспортних засобах можуть їздити наступні види пасажирів: звичайний пасажир, пожежник, поліцейський. Ієрархія вказаних об'єктів подана на рис. 1. Додайте в меню команду для збереження пасажирів в файл та передбачте можливість читання набору пасажирів з файлу (використовуючи `ObjectOutputStream` та `ObjectInputStream`).

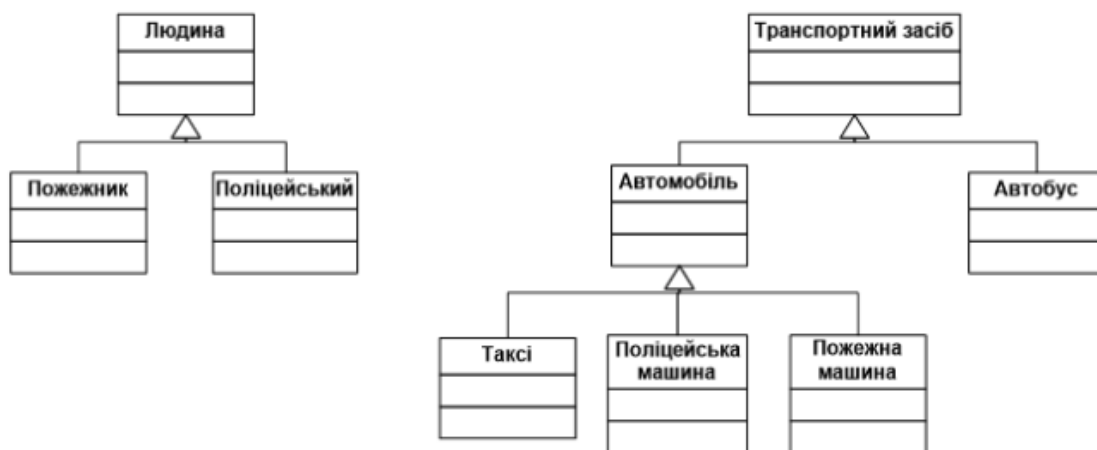


Рис. 1.

Автобус та таксі можуть перевозити будь-яких пасажирів, пожежна машина – тільки пожежників, міліцейська машина – тільки міліціонерів. Реалізувати на основі узагальненого програмування (generics) вказані обмеження щодо перевезки пасажирів.

Для класів транспортних засобів реалізувати наступні функції:

- Кожний транспортний засіб має обмежену кількість місць. Реалізувати функцію для отримання максимальної кількості місць та функцію для отримання кількості зайнятих місць.
- Посадка пасажирів у транспортний засіб. Якщо всі місця вже зайнято, функція повинна ініціювати виключну ситуацію.
- Висадка пасажирів із транспортного засобу. Функція повинна ініціювати виключну ситуацію, якщо вказаний пасажир «не сидить» у транспортному засобі.

Додатково реалізувати функцію підрахунку кількості людей, які перевозяться на автомобілях на дорозі. Варіант коду наданий нижче. Дописати код до працездатного. Обов'язково використовувати generics та wildcard (де це потрібно).

```
public class Road { public List carsInRoad = new ArrayList<>();  
public int getCountOfHumans(){....}  
public void addCarToRoad( .... ){ ... } }
```

Реалізувати модульні тести з наповнення транспортних засобів різними типами пасажирів (не забувайте і про тестування виключних ситуацій).

Вирішення завдання

У цій лабораторній роботі я працювала над реалізацією системи перевезень, використовуючи принципи узагальненого програмування (Generics). Моїм основним завданням було не просто створити класи для машин і пасажирів, а налаштувати їх так, щоб сама мова Java контролювала правила посадки.

Як я побудувала ієрархію: Спочатку я створила базовий клас Human і зробила його серіалізованим (Serializable), щоб у майбутньому мати змогу зберігати дані у файл. Від нього я успадкувала спеціалізовані типи: Fireman (пожежник) та Policeman (поліцейський).

Для транспортних засобів я використала абстрактний клас Vehicle<T extends Human>. Це був ключовий момент: завдяки параметру <T> я змогла чітко обмежити, хто може їздити в конкретній машині. Наприклад, для пожежної машини я вказала тип Fireman, тому спроба посадити туди звичайну людину призведе до помилки ще на етапі написання коду.

Логіка роботи програми:

Посадка та обмеження: У кожному транспорті я передбачила обмежену кількість місць. Я написала метод addPassenger, який перевіряє, чи є вільні крісла. Якщо машина заповнена, я ініціюю виключну ситуацію (Exception), як того вимагає завдання. Аналогічну логіку я додала і для висадки — програма видасть помилку, якщо ми спробуємо висадити пасажирів, якого немає всередині.

Облік на дорозі: Для підрахунку всіх людей на дорозі я створила клас Road. Тут я використала wildcard (Vehicle<?>), що дозволило мені зберігати в одному списку і автобуси, і таксі, і поліцейські авто одночасно. Метод getCountOfHumans просто "пробігає" по всіх машинах і сумує кількість зайнятих місць.

Робота з файлами: Для виконання пункту про збереження, я реалізувала метод saveToFile. Використовуючи ObjectOutputStream, я

записую список пасажирів у бінарний файл data.ser. Це дозволяє повністю зберегти стан об'єктів.

```
// Базовий клас Людина
public class Human implements Serializable {
    private String name;
    public Human(String name) { this.name = name; }
    public String getName() { return name; }
}

// Пожежник
class Fireman extends Human {
    public Fireman(String name) { super(name); }
}

// Поліцейський
class Policeman extends Human {
    public Policeman(String name) { super(name); }
}
```

```
public class Road {
    public List<Vehicle<?>> carsInRoad = new ArrayList<>();

    // Підрахунок всіх людей на дорозі
    public int getCountOfHumans() {
        int count = 0;
        for (Vehicle<?> car : carsInRoad) {
            count += car.getOccupiedSeats();
        }
        return count;
    }

    // Додавання машини на дорогу
    public void addCarToRoad(Vehicle<?> car) {
        carsInRoad.add(car);
    }
}
```

```
// Транспортний засіб
public abstract class Vehicle<T extends Human> {
    private final int maxSeats;
    private final List<T> passengers = new ArrayList<>();

    public Vehicle(int maxSeats) { this.maxSeats = maxSeats; }

    public int getMaxSeats() { return maxSeats; }
    public int getOccupiedSeats() { return passengers.size(); }

    // Посадка пасажирів
    public void addPassenger(T passenger) throws Exception {
        if (passengers.size() >= maxSeats) {
            throw new Exception("Всі місця вже зайнято!");
        }
        passengers.add(passenger);
    }

    // Висадка пасажирів
    public void removePassenger(T passenger) throws Exception {
        if (!passengers.contains(passenger)) {
            throw new Exception("Вказаний пасажир не знаходиться в транспорті!");
        }
        passengers.remove(passenger);
    }
}
```

```
// Автобус: може перевозити будь-яких пасажирів
class Bus extends Vehicle<Human> {
    public Bus(int seats) { super(seats); }
}

// Таксі: може перевозити будь-яких пасажирів
class Taxi extends Vehicle<Human> {
    public Taxi(int seats) { super(seats); }
}

// Пожежна машина: тільки пожежників
class FireTruck extends Vehicle<Fireman> {
    public FireTruck(int seats) { super(seats); }
}

// Поліцейська машина: тільки поліцейських
class PoliceCar extends Vehicle<Policeman> {
    public PoliceCar(int seats) { super(seats); }
}
```

```
public class MainApp {
    public static void main(String[] args) {
        try {
            System.out.println("— Етап 1: Ініціалізація транспорту та людей —");
            // Створюємо транспорт
            Bus bus = new Bus(20);
            FireTruck fireTruck = new FireTruck(4);
            System.out.println("Створено автобус (20 місць) та пожежну машину (4 місця) ");

            // Створюємо людей
            Human person = new Human("Іван");
            Fireman fireman = new Fireman("Степан");
            System.out.println("Створено пасажирів: " + person.getName() + " та " + fireman.getName());

            System.out.println("\n— Етап 2: Тестування посадки пасажирів —");
            // Тестуємо посадку
            bus.addPassenger(person);
            System.out.println("Іван сів в автобус. Зайнято місць: " + bus.getOccupiedSeats() + " ");

            fireTruck.addPassenger(fireman);
            System.out.println("Степан (пожежник) сів в пожежну машину. Зайнято місць: " + fireTruck.getOccupiedSeats());

            System.out.println("\n— Етап 3: Робота на дорозі (Wildcards) —");
            // Демонстрація роботи Road
            Road road = new Road();
            road.addCarToRoad(bus);
            road.addCarToRoad(fireTruck);
            System.out.println("Машини виїхали на дорогу");
            System.out.println("Загальна кількість людей на дорозі: " + road.getCountOfHumans() + );

            System.out.println("\n— Етап 4: Сериалізація даних в файл —");
            // Збереження списку пасажирів у файл
            List<Human> passengers = new ArrayList<>();
            passengers.add(person);
            passengers.add(fireman);

            System.out.println("Підготовка до збереження списку з " + passengers.size() + " людей в файл...");
            saveToFile(passengers, "data.ser");
            System.out.println("Дані успішно збережено в файл 'data.ser' ");

        } catch (Exception e) {
            // Обробка виключних ситуацій (переповнення місць тощо)
            System.err.println("\n[ПОМИЛКА]: " + e.getMessage());
        }
    }
}
```

```

public static void saveToFile(List<Human> data, String filename) throws IOException {
    try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(filename))) {
        oos.writeObject(data);
    }
}

```

Коли я запускаю програму, у терміналі чітко відображається кожен мій крок. Я спеціально додала детальні логи, щоб було видно які об'єкти створені, хто саме сів у автобус чи пожежну машину, як Road підраховує загальну кількість людей незалежно від типу машини, момент успішного запису даних у файл.

```

— Етап 1: Ініціалізація транспорту та людей —
Створено автобус (20 місць) та пожежну машину (4 місця)
Створено пасажирів: Іван та Степан

— Етап 2: Тестування посадки пасажирів —
Іван сів у автобус. Зайнято місць: 1
Степан (пожежник) сів у пожежну машину. Зайнято місць: 1

— Етап 3: Робота на дорозі (Wildcards) —
Машини виїхали на дорогу [cite: 24]
Загальна кількість людей на дорозі: 2 [cite: 25]

— Етап 4: Серіалізація даних у файл —
Підготовка до збереження списку з 2 людей у файл...
Дані успішно збережено у файл 'data.ser'

```

Висновок

Ця робота допомогла мені краще розібратися з тим, як працюють узагальнення в Java. Я навчилася використовувати wildcards для створення гнучких колекцій і зрозуміла, як серіалізація допомагає зберігати дані об'єктів поза пам'яттю програми.